1. Desktop Computers: Architecture & OS Review

Operating Systems (OS)

Master in Innovation and Research in Informatics (MIRI)

Facultat d'Informàtica de Barcelona (FIB)

Universitat Politècnica de Catalunya (UPC)

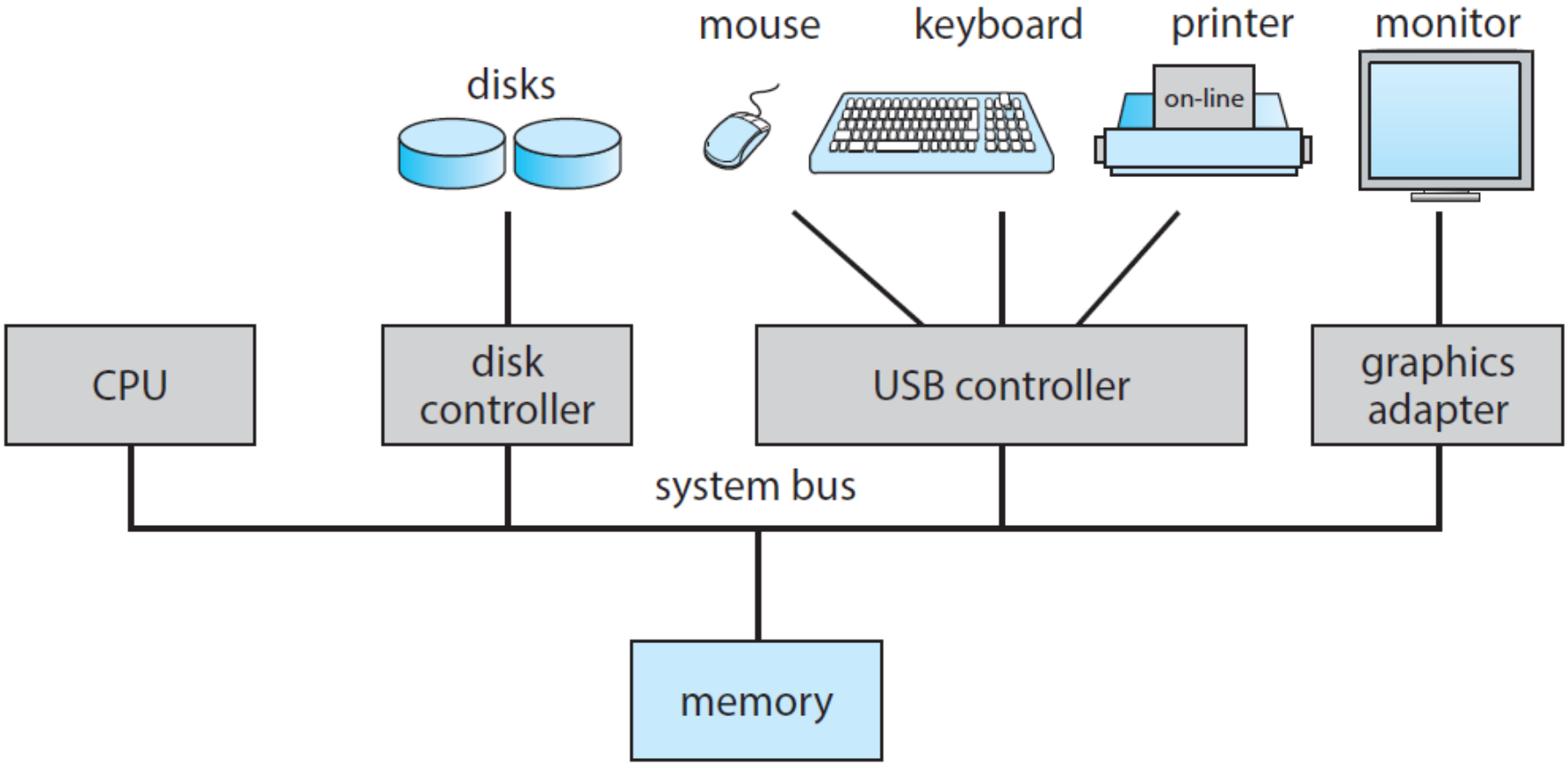
2025/2026 Q1

FIB

Contents

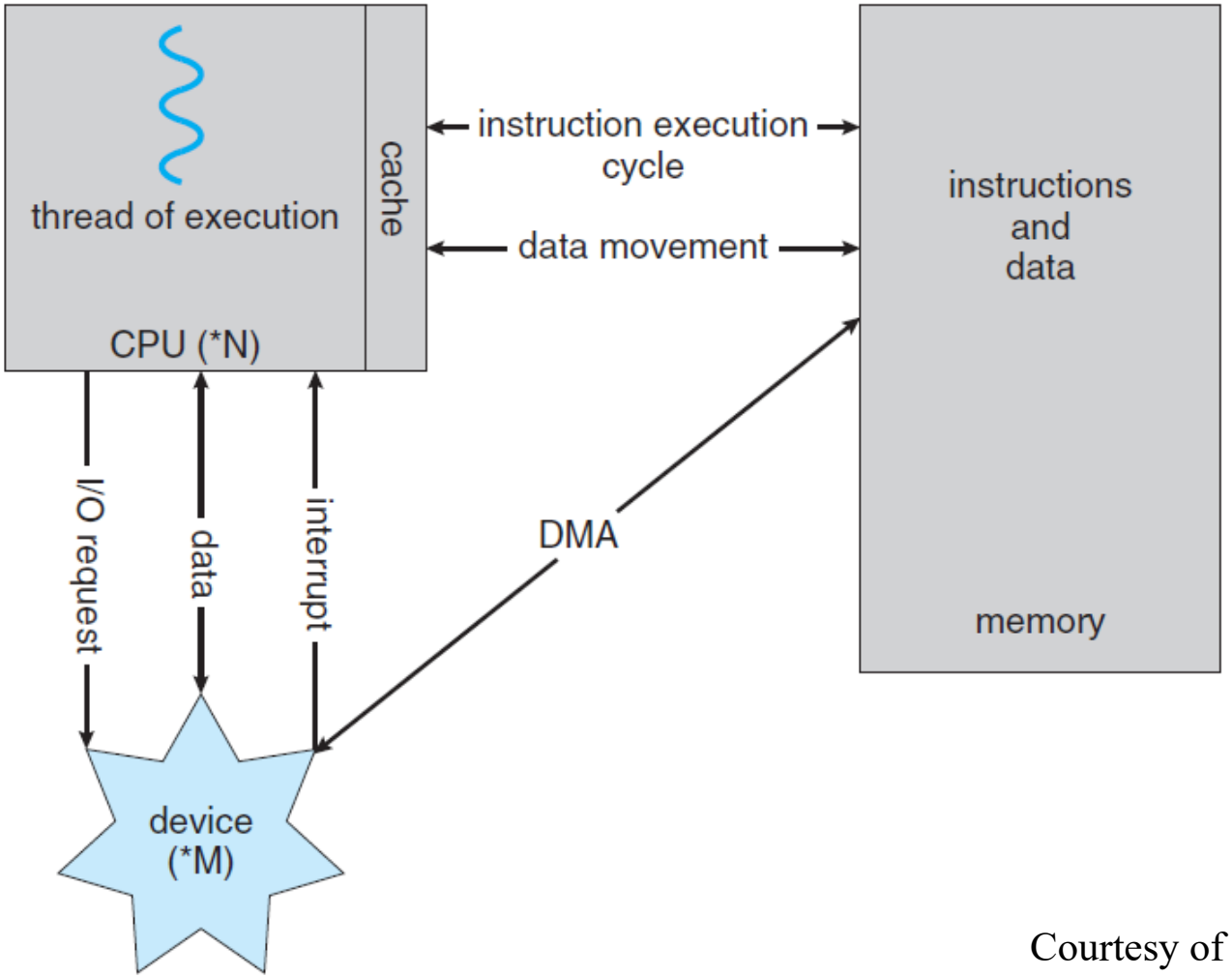
- **Computer architecture**
- Operating system functionality
- Access to the OS
- Memory and address spaces
- Execution of user programs
- Processes
- Mass storage and file systems

Computer system architecture



Courtesy of [SiGG18]

Interplay of components



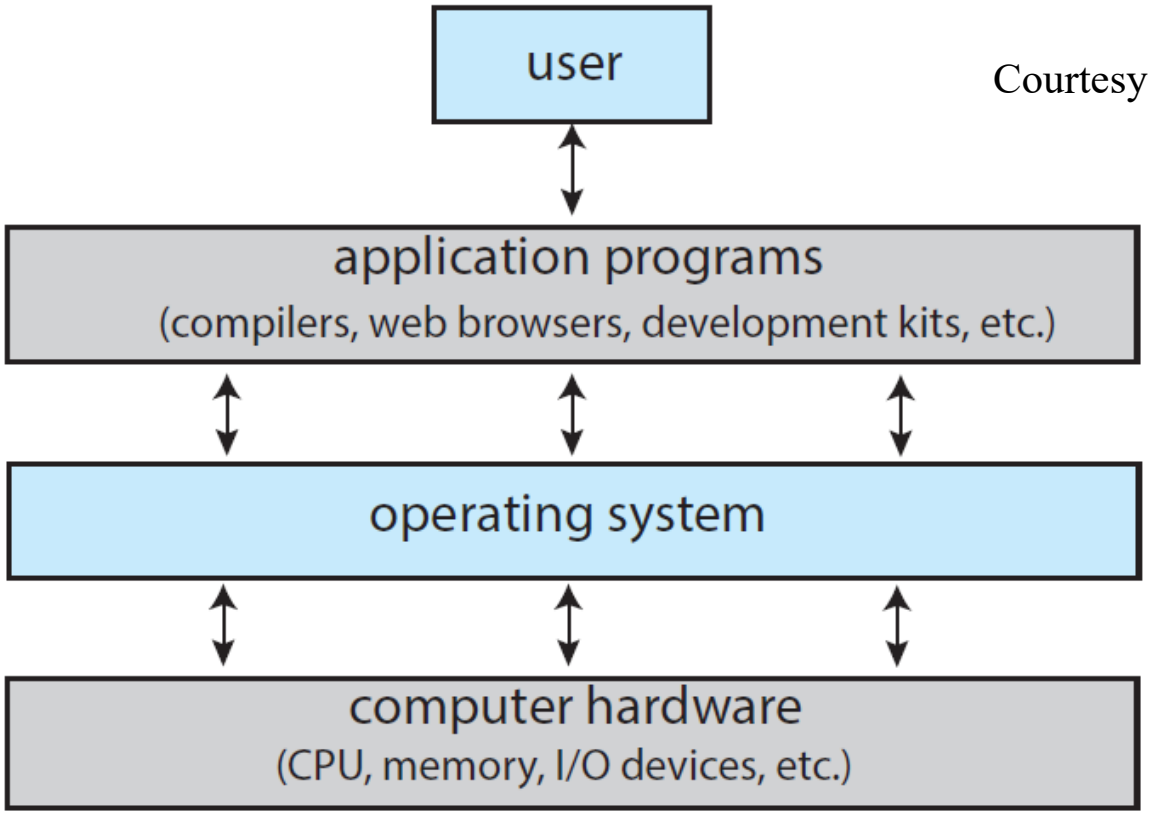
Courtesy of [SiGG18]

Contents

- Computer architecture
- **Operating system functionality**
- Access to the OS
- Memory and address spaces
- Execution of user programs
- Processes
- Mass storage and file systems

Operating system functionality

- SW layer between users and HW grouping common functions for controlling and allocating resources

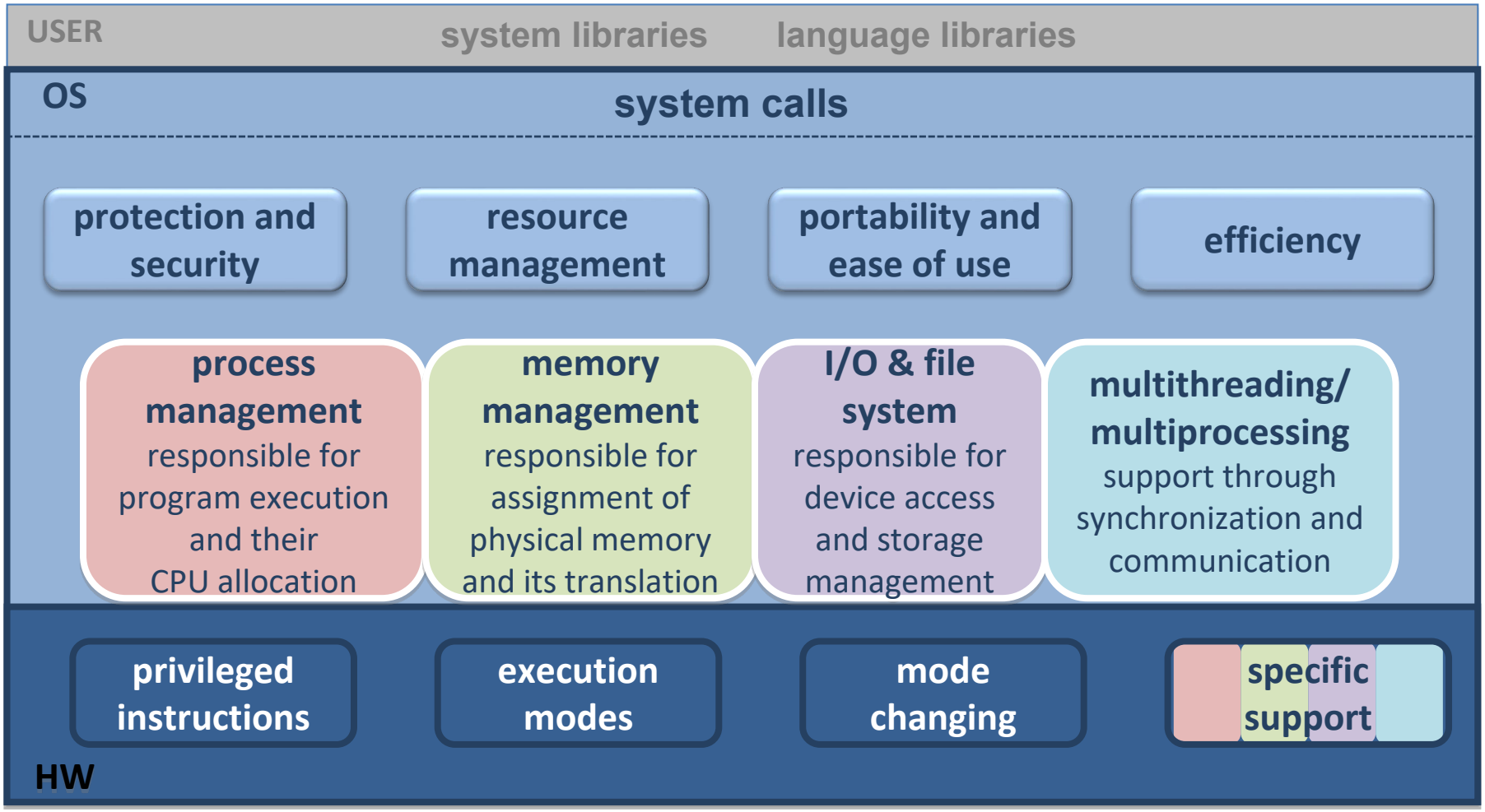


Courtesy of [SiGG18]

Operating system functionality

- OS acts as a control program
  - Abstract users from low-level HW-dependent code
  - Control the various I/O devices and user programs
  - Prevent errors, improper use, and damage to the computer and other users
- OS acts as a resource allocator
  - Deal with numerous and possibly conflicting requests for resources
  - Arbitrate resource allocation to programs
  - Operate the computer efficiently and fairly

Operating system features



Contents

- Computer architecture
- Operating system functionality
- **Access to the OS**
- Memory and address spaces
- Execution of user programs
- Processes
- Mass storage and file systems

Access to the OS

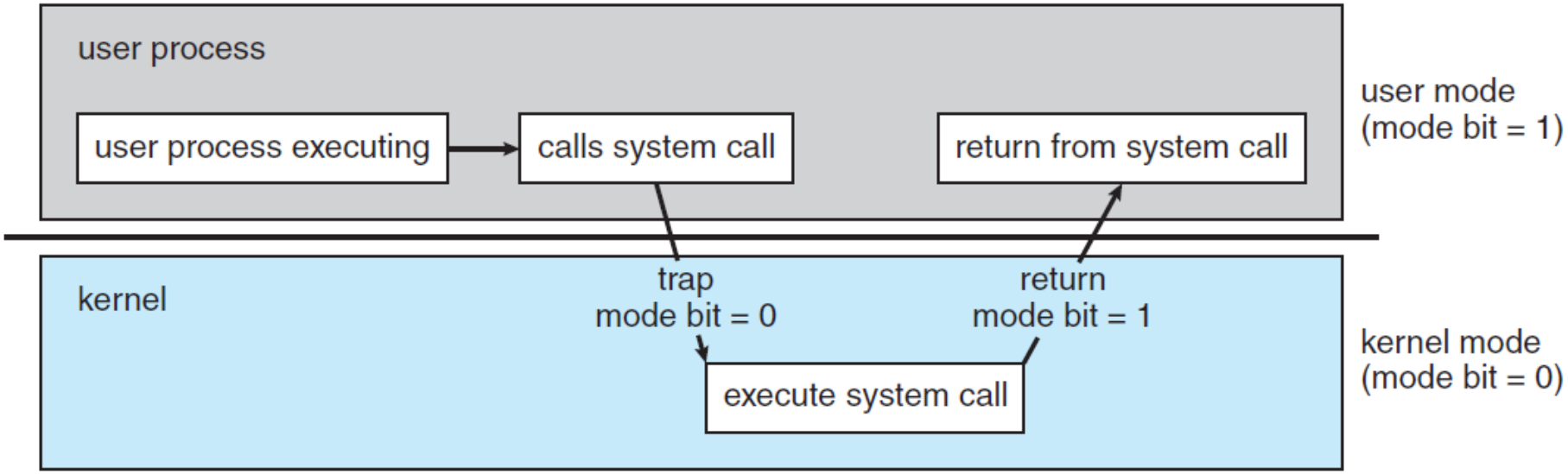
- Modern operating systems are **event driven**
- Events are usually signaled by an interrupt from either the hardware or the software
- For each type of interrupt, the OS provides a service routine to deal with it
- Service routines contain sensitive instructions changing the underlying HW configuration
- Sensitive instructions should be only allowed when executing OS code
  - HW support is necessary!

Access to the OS

- HW should differentiate among various modes of operation (at least 2)
  - **Kernel mode** vs. **User mode**
- HW allows some instructions (i.e. privileged) to be executed only in kernel mode
  - If a privileged instruction is attempted in user mode, an exception is generated
- Ideally, sensitive instructions should be a subset of the privileged instructions

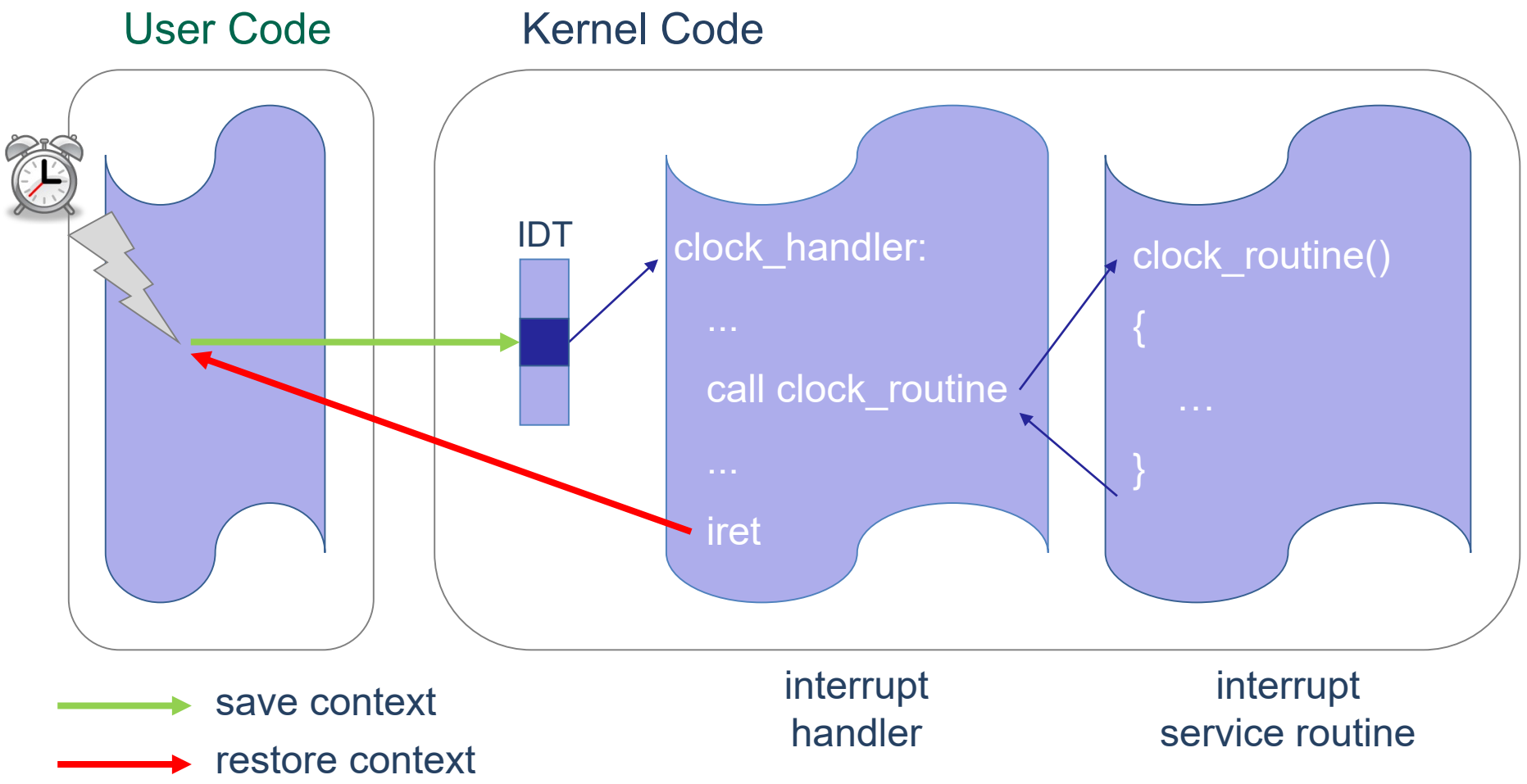
Access to the OS

- HW transitions automatically from user to kernel mode (and vice versa) when interrupts and traps happen

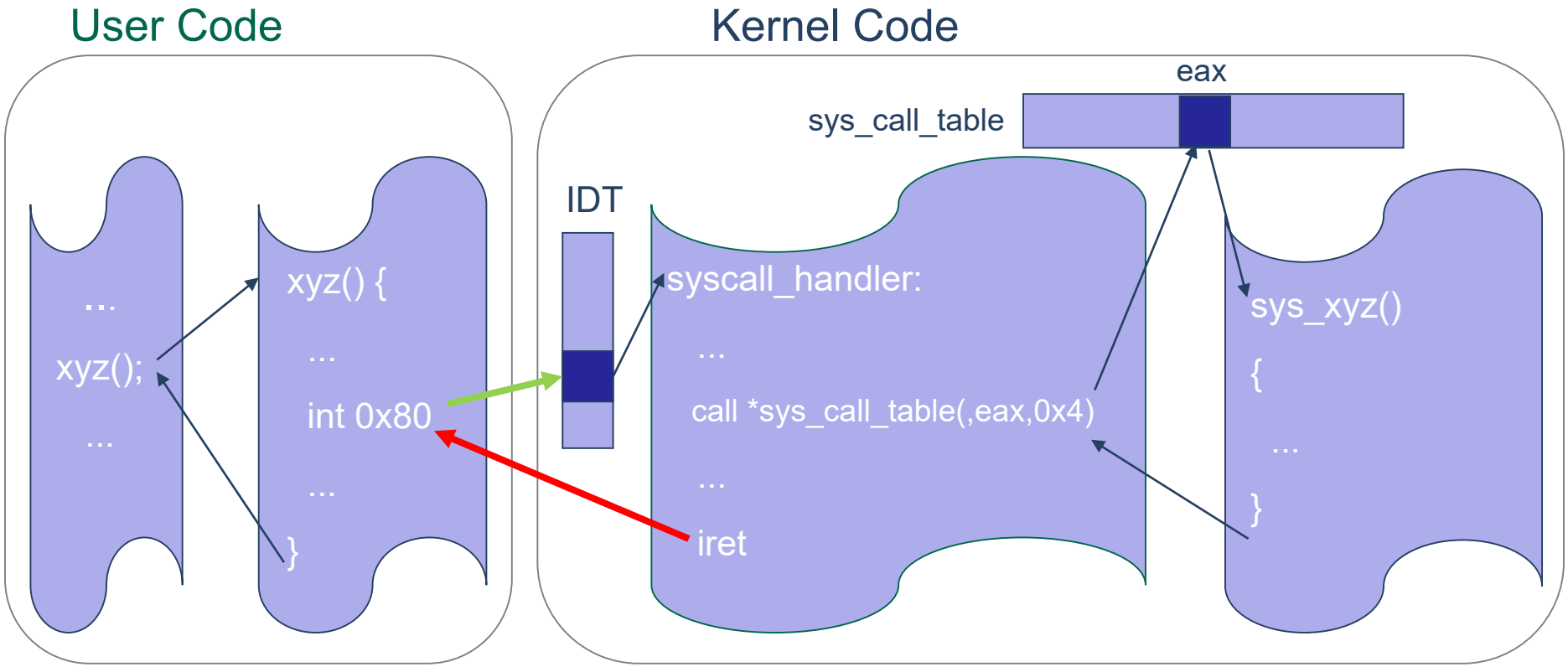


Courtesy of [SiGG18]

Example of privilege escalation



User-initiated privilege escalation



system call invocation in application program

wrapper for system call

system call handler

system call service routine

Contents

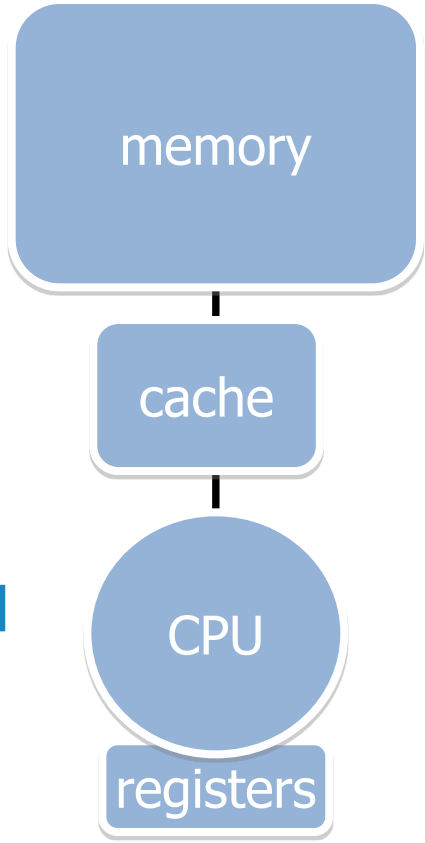
- Computer architecture
- Operating system functionality
- Access to the OS
- **Memory and address spaces**
- Execution of user programs
- Processes
- Mass storage and file systems

Memory

- Memory is a direct-access large array of bytes
- Each byte can be referenced from a program through its own address
- Memory is accessed with specific instructions
  - Load
  - Store
- Completing a memory access may take many cycles of the CPU clock
  - ~83 ns (250 cycles) [Duar08]

Cache memory

- Add cache to speed up memory access without any OS intervention
  - CPU reads from cache instead of RAM
    - L1: ~1 ns (3 cycles) [Duar08]
    - L2: ~4.7 ns (14 cycles) [Duar08]
    - RAM: ~83 ns (250 cycles) [Duar08]
  - Cache keeps last accessed data
    - Exploit temporal locality
  - Entire cache lines are loaded from RAM
    - Exploit spatial locality
  - HW does all the work



Memory hierarchy

- We can differentiate various memory layers according to their **speed**, **capacity**, and **cost** tradeoff

Level	1	2	3	4	5
Name	registers	cache	main memory	solid-state disk	magnetic disk
Typical size	< 1 KB	< 16MB	< 64GB	< 1 TB	< 10 TB
Implementation technology	custom memory with multiple ports CMOS	on-chip or off-chip CMOS SRAM	CMOS SRAM	flash memory	magnetic disk
Access time (ns)	0.25-0.5	0.5-25	80-250	25,000-50,000	5,000,000
Bandwidth (MB/sec)	20,000-100,000	5,000-10,000	1,000-5,000	500	20-150
Managed by	compiler	hardware	operating system	operating system	operating system
Backed by	cache	main memory	disk	disk	disk or tape

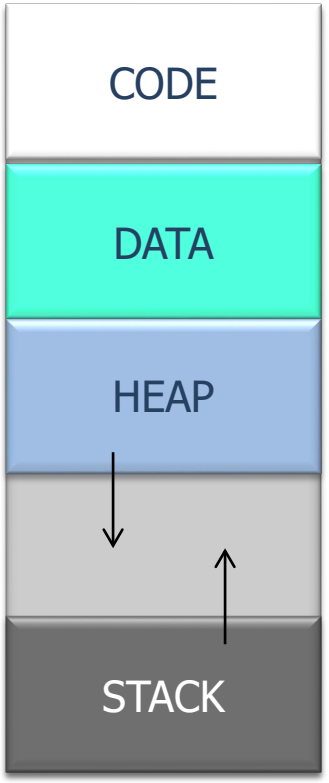
Courtesy of [SiGG18]

Address spaces

- Memory offers a single linear address space, but must keep several programs concurrently
  - Logical process address space
    - The set of all the addresses generated by a process during its execution
    - It is RELATIVE to the process and independent of the current assignment of data to memory
  - Physical process address space
    - Actual locations in the main memory associated with the logical addresses
    - A run-time **mapping from logical to physical addresses** is needed

Logical process address space

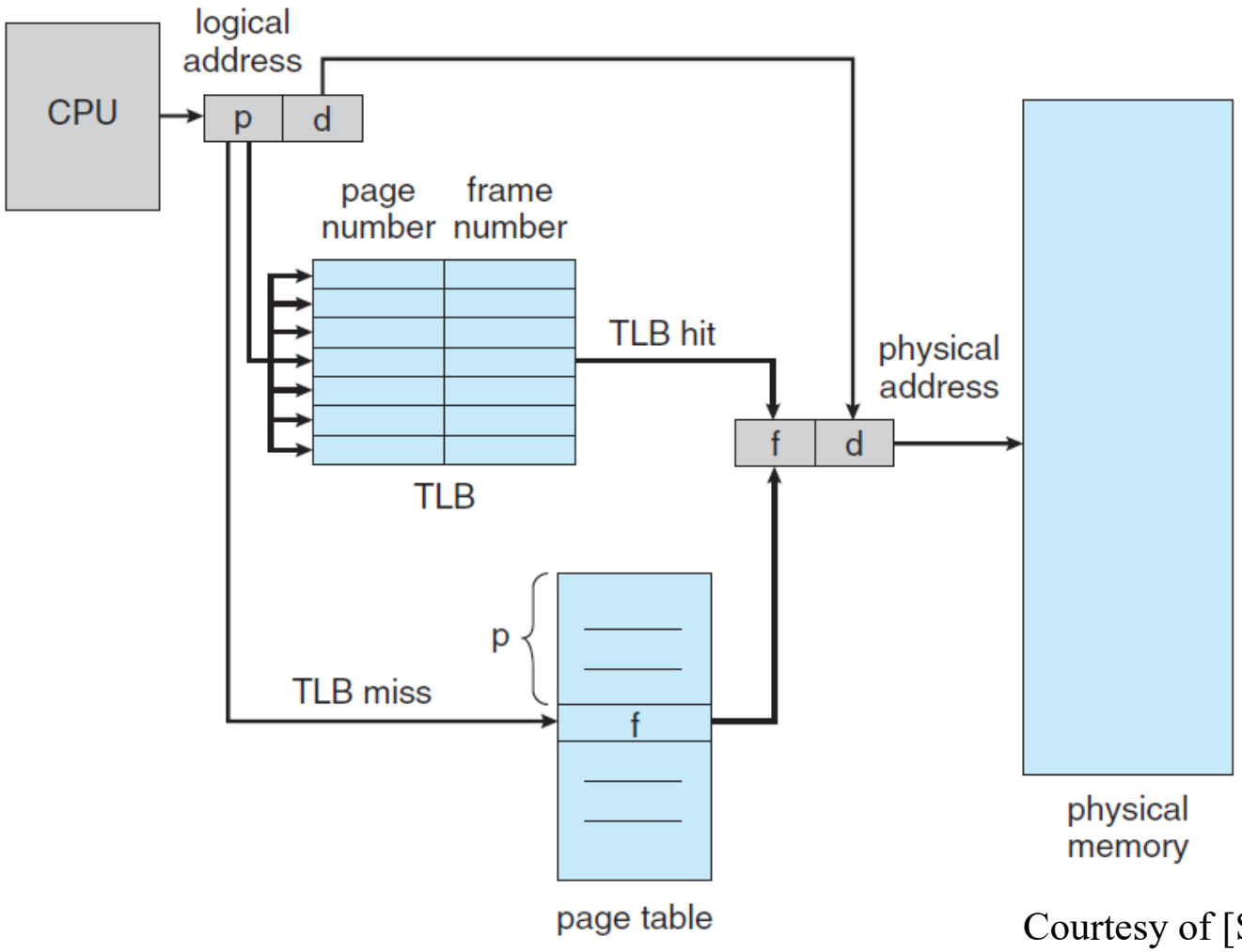
- Sections:
  - Code
  - Data (initialized and un-initialized)
  - Stack
  - Heap (for dynamic memory)
- Addresses start at address 0
  - For ALL the processes!
- Physically mapped somewhere else
  - Mapping is supported by the HW
    - Via the **Memory Management Unit (MMU)**



Paging

- Most OS use **paging** to do the mapping
- Physical memory is broken into fixed-sized blocks called frames and logical memory into blocks of the same size called pages
- A page table per process contains the frame of each of its pages in physical memory
- Page tables are kept in memory, and a base register (PTBR) points to the current one
- Translation is speeded up using a small, fast-lookup **hardware cache** called a Translation Look-aside Buffer (TLB)

Paging



Courtesy of [SiGG18]

Memory management

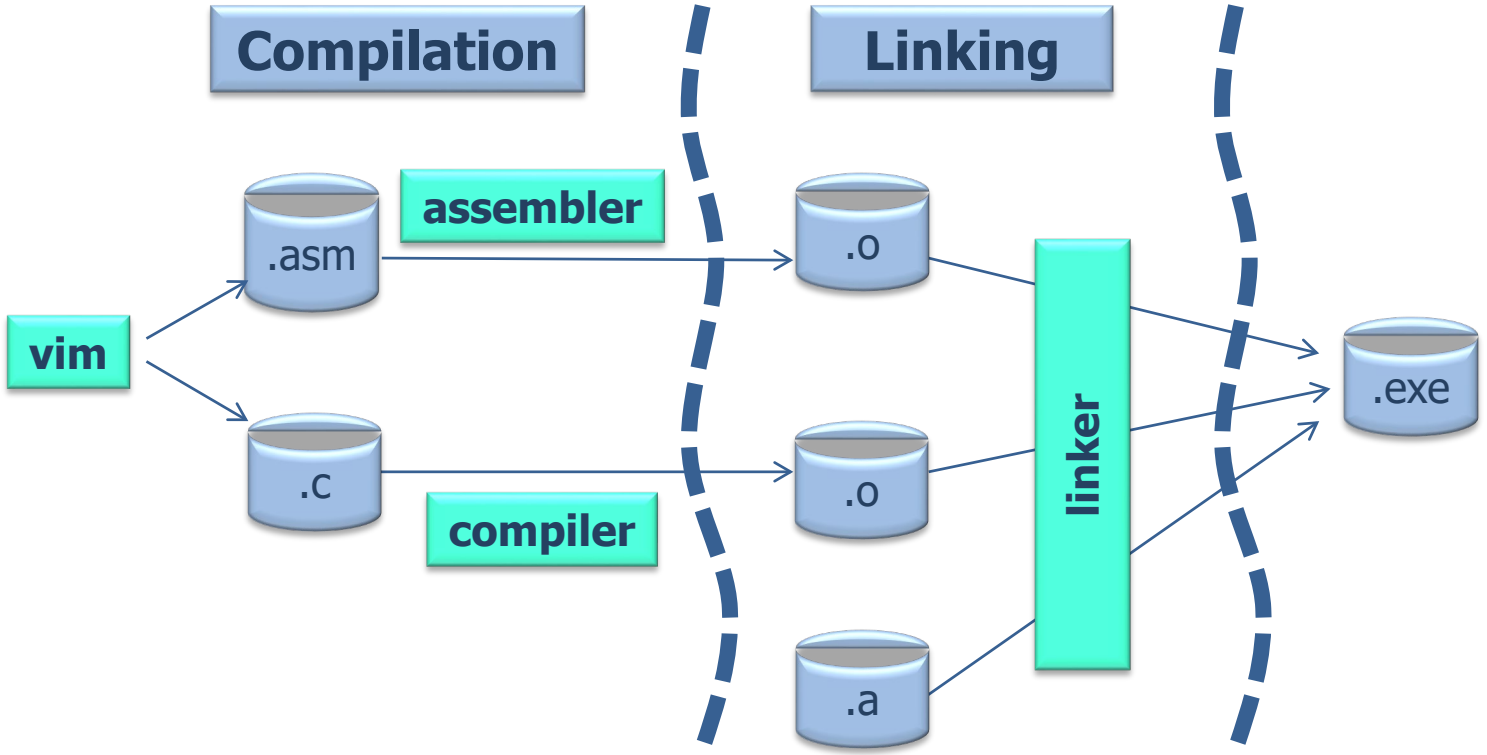
- OS is responsible for the following activities in connection with memory management:
  - Keeping track of which parts of memory are currently being used and who is using them
  - Allocating and deallocating memory space as needed
  - Deciding which processes (or parts of processes) and data to move into and out of memory

Contents

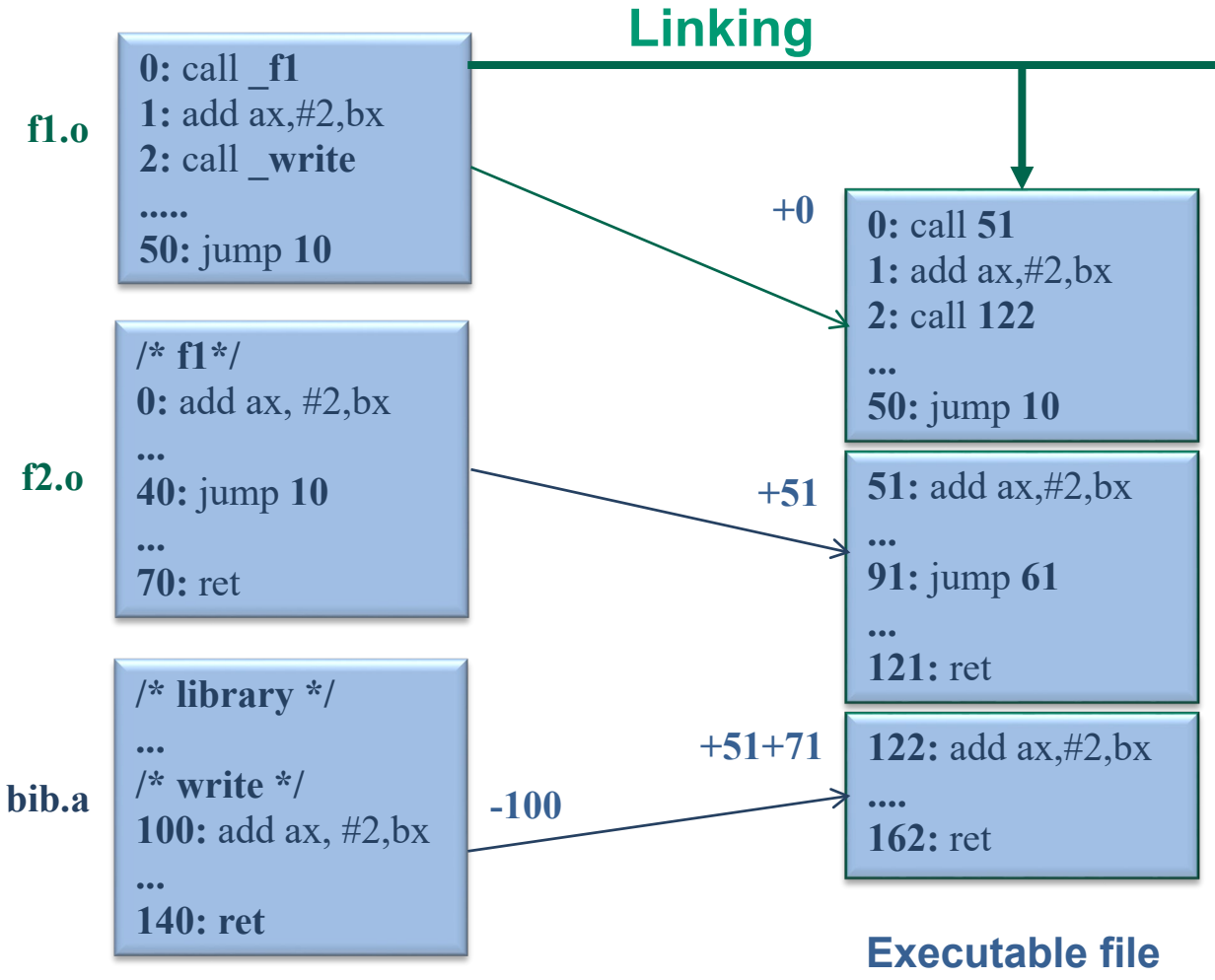
- Computer architecture
- Operating system functionality
- Access to the OS
- Memory and address spaces
- **Execution of user programs**
- Processes
- Mass storage and file systems

Program loading and execution

1. Generate a file suitable for execution



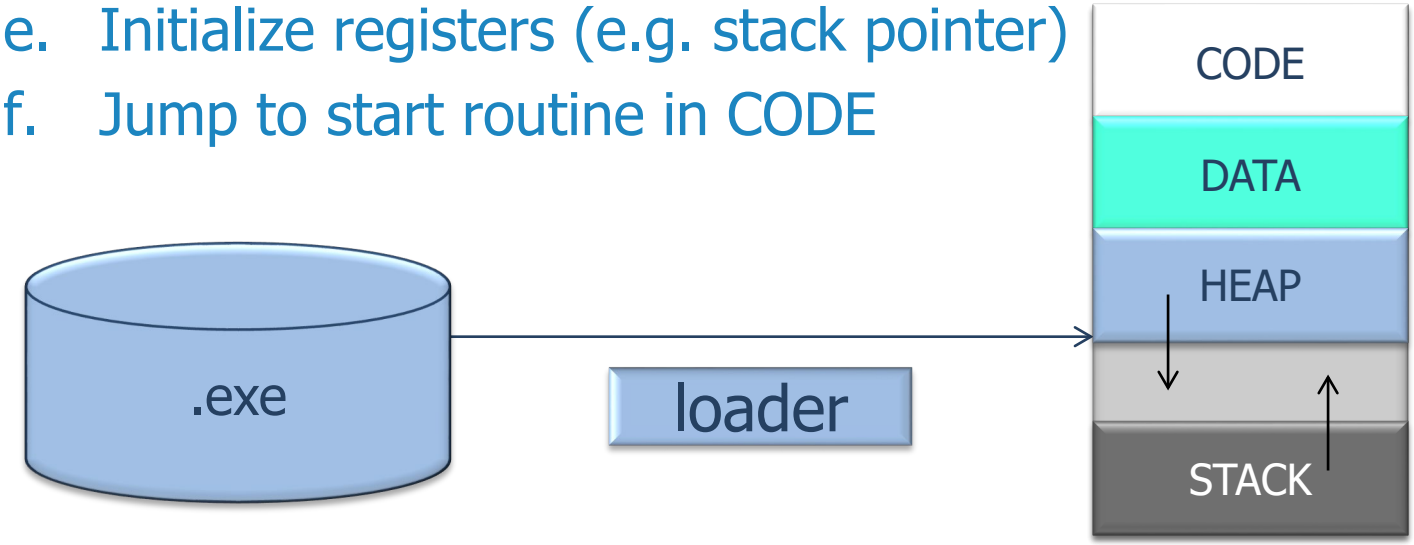
Compilation & Linking



1. Resolve references to external symbols
2. Assign final addresses to procedures and variables
3. Revise code and data to reflect new addresses

Program loading

2. Load .exe file into memory and run program
   a. Parse and validate the executable file
   b. Allocate memory for program execution
   c. Load code and data sections into memory
   d. Copy program arguments onto the stack
   e. Initialize registers (e.g. stack pointer)
   f. Jump to start routine in CODE



Contents

- Computer architecture
- Operating system functionality
- Access to the OS
- Memory and address spaces
- Execution of user programs
- **Processes**
- Mass storage and file systems

Process concept

- A **process** is a user program in execution. It is the OS unit of work consisting of:
  1. Sequence of instructions to be executed
  2. Current state (context)
     - Values of the program counter, status registers, stack and frame pointers, and CPU general-purpose registers
  3. Set of associated resources and control information to use them
     - Memory (code, data, process stack, heap)
     - I/O devices (open files, …)
     - Process ID, state, and scheduling parameters
     - Accounting information

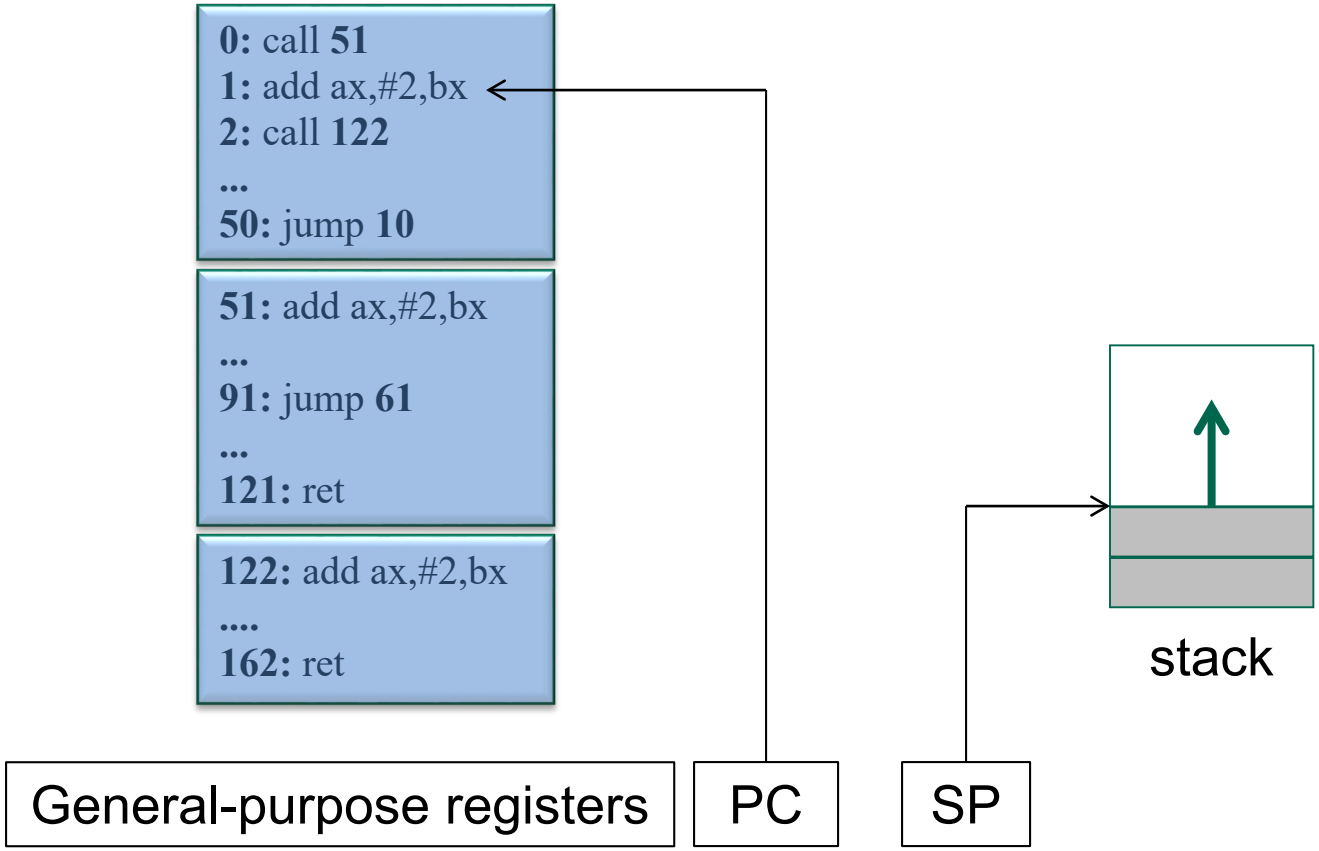
Process: sequence of instructions

```
0: call 51
1: add ax,#2,bx
2: call 122
...
50: jump 10
```

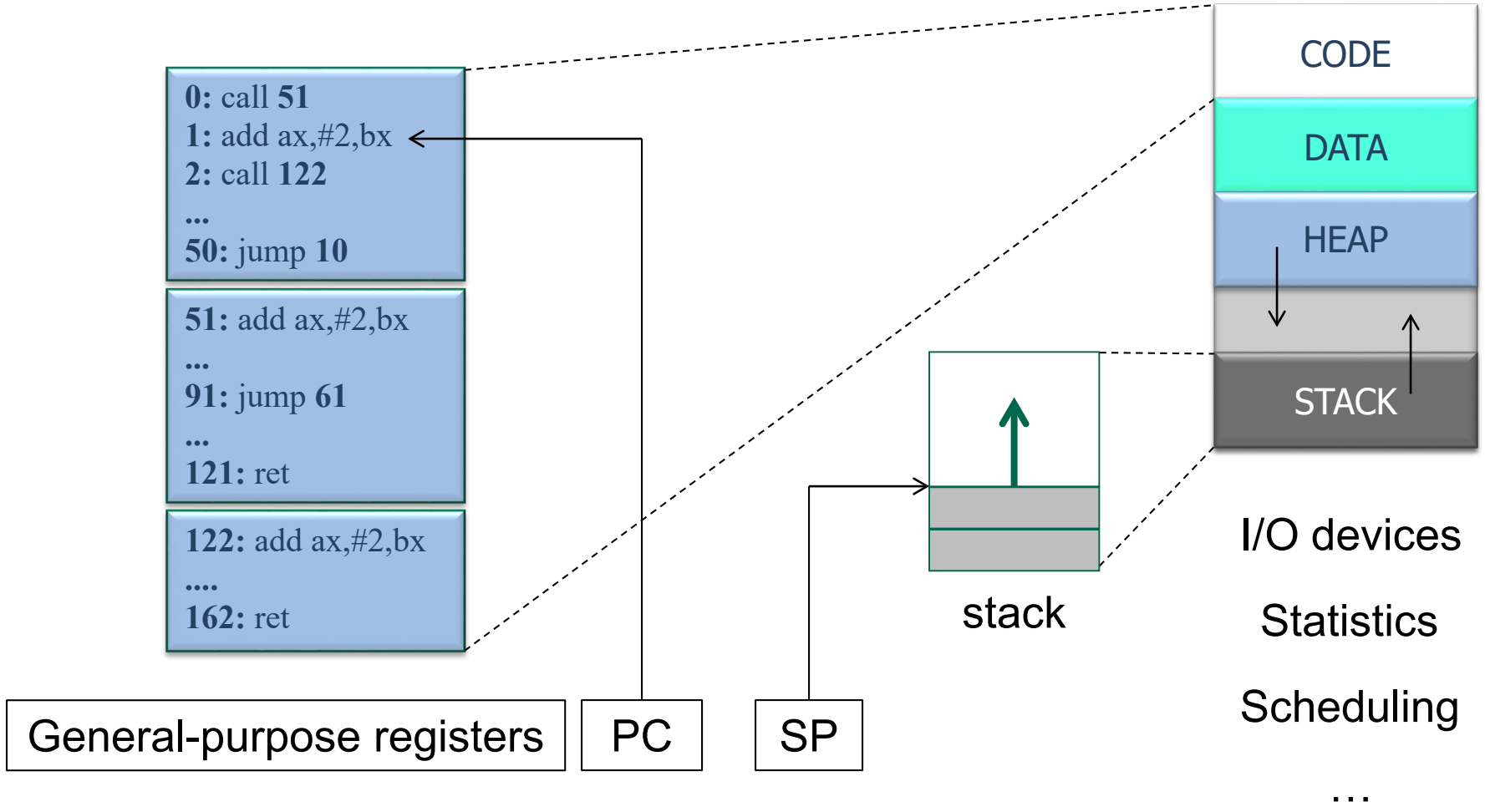
```
51: add ax,#2,bx
...
91: jump 61
...
121: ret
```

```
122: add ax,#2,bx
....
162: ret
```

Process: current state



Process: associated resources

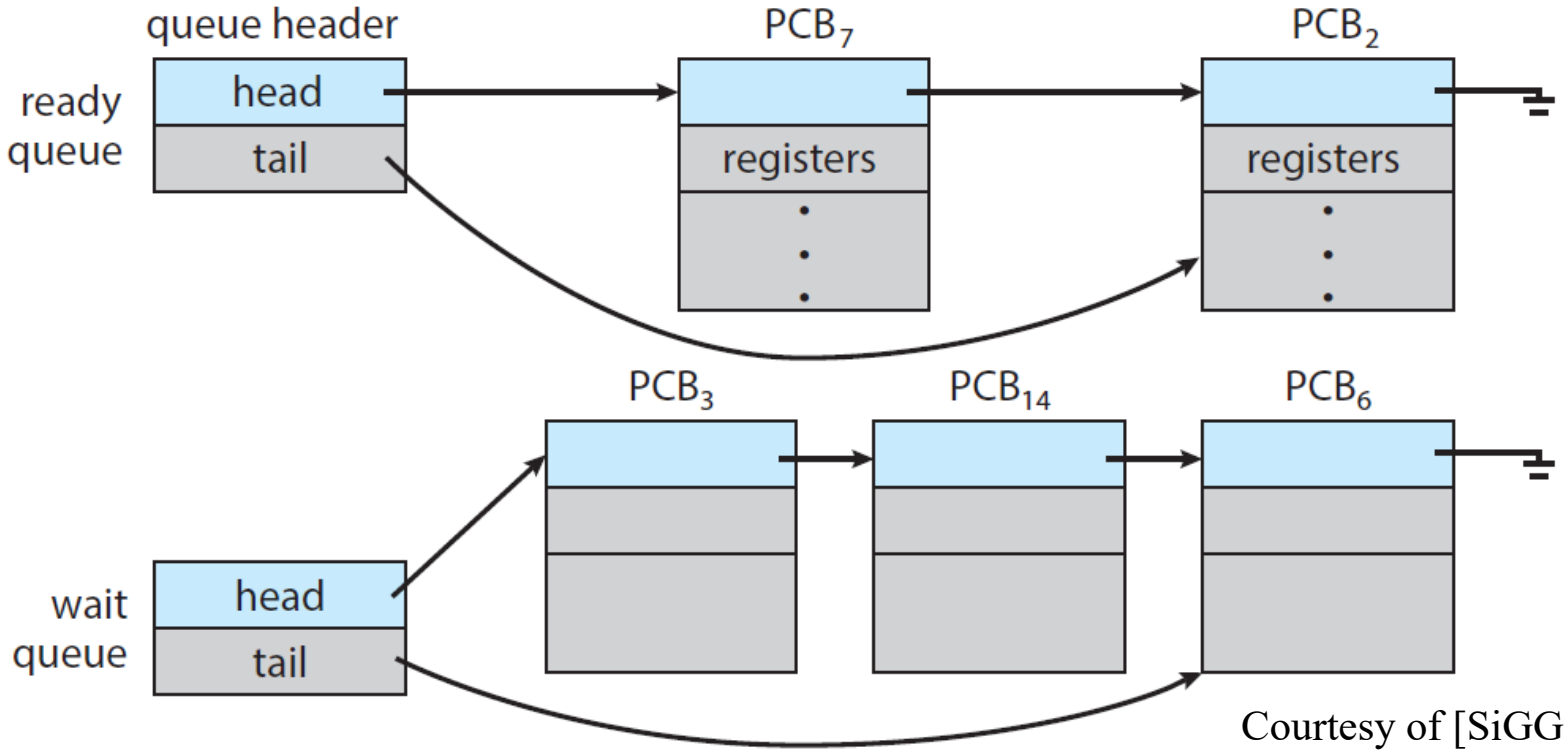


Process: associated resources

- From the user's viewpoint, different processes do not share their associated resources
  - Unless explicitly requested to the OS
    - e.g. shared memory, threads
- However, resources are limited and OS must arbitrate their shared use among processes
  - OS **virtualizes** the resources to give the illusion that each process has dedicated access to them
    - i.e. OS separates the logical view of the resource (dedicated) from the physical one (potentially shared)
  - e.g. execution on the CPU, access to I/O devices

Process scheduling

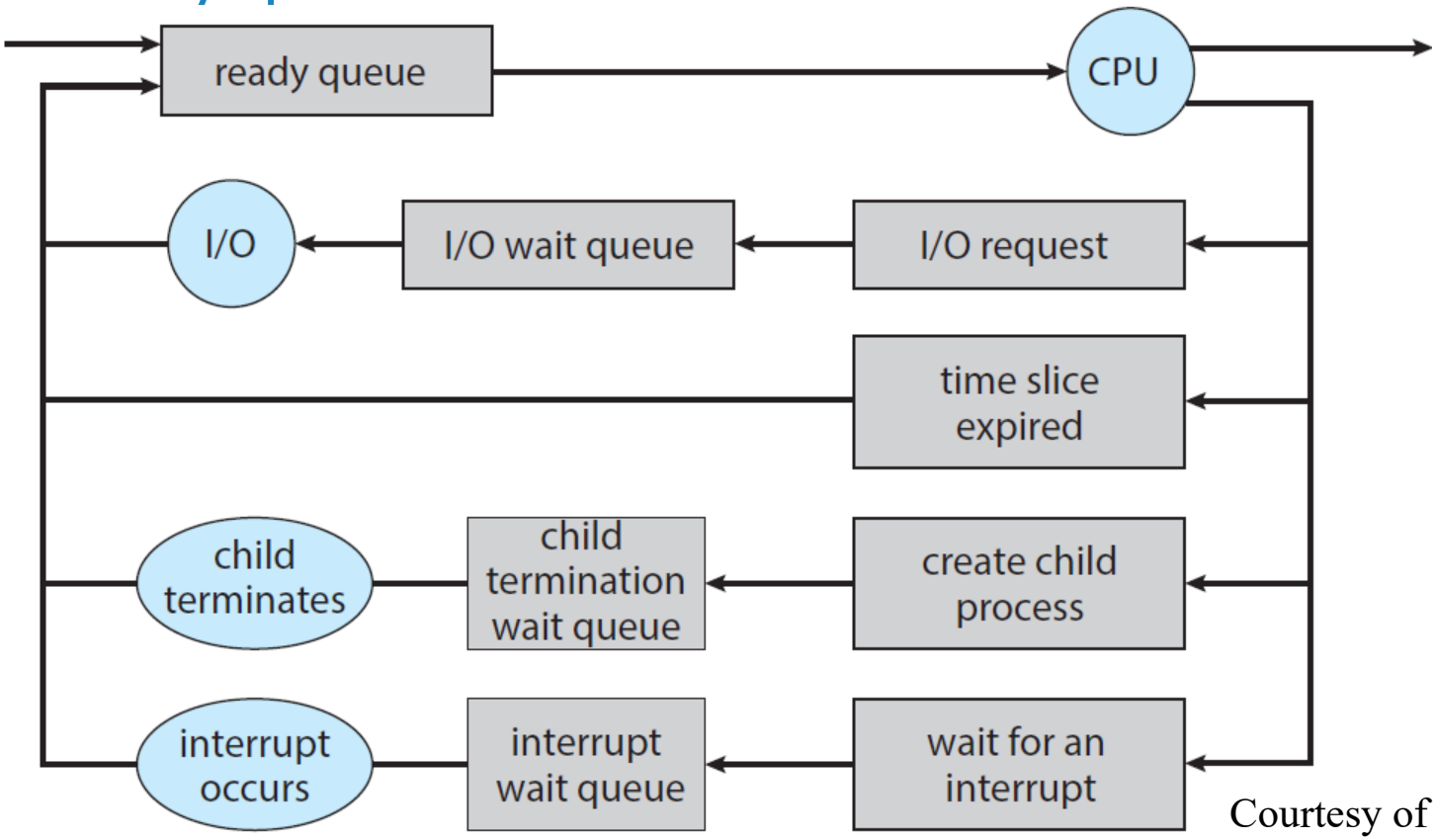
- A process migrates among the ready queue (to run on the CPU) and various wait queues (for certain events to occur) throughout its lifetime



Courtesy of [SiGG18]

Process scheduling

- OS applies a policy to select **which** process from the ready queue should be run next in the CPU



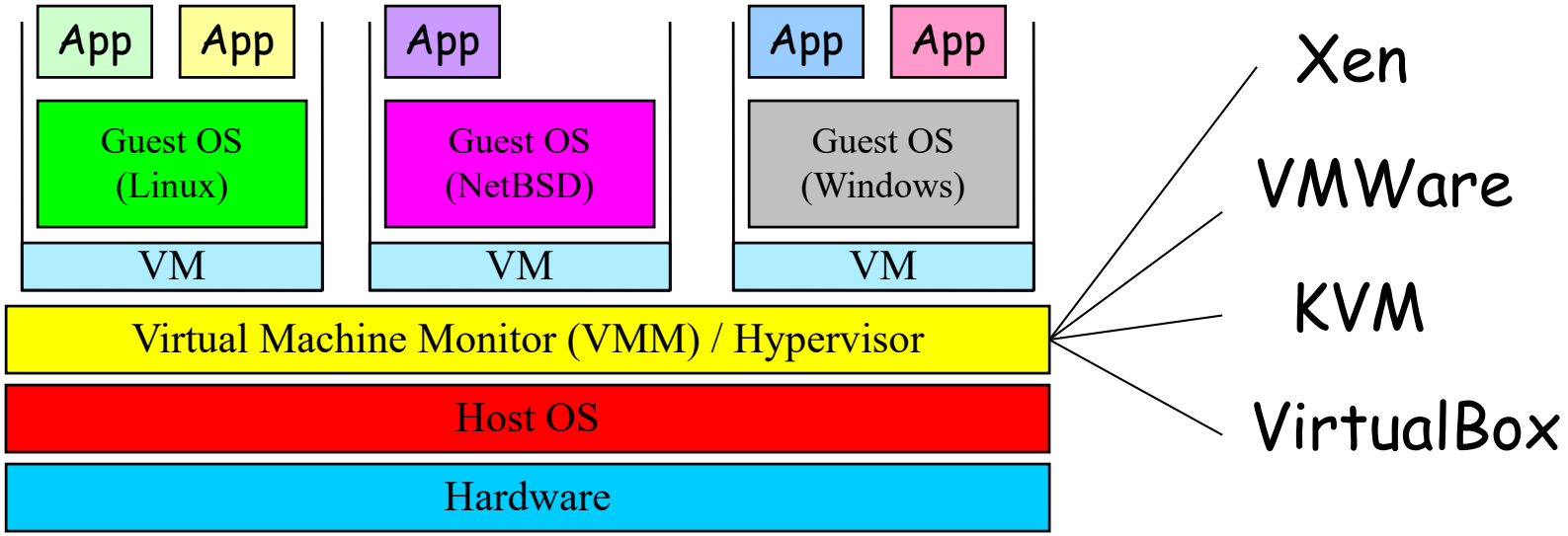
Courtesy of [SiGG18]

Process management

- OS is responsible for the following activities in connection with process management:
  - Creating and deleting processes
  - Suspending and resuming processes
  - Scheduling processes and threads on the CPUs
  - Providing mechanisms for process synchronization
  - Providing mechanisms for process communication

Virtualization

- Go beyond virtualization of individual resources and abstract a complete system (HW: CPU, RAM, disk, … + SW: OS, Apps) into a dedicated virtual execution environment
  - Multiple virtual machines can run on a single host



Virtualization

- Pros
  - Security & performance isolation for programs
  - Protect underlying HW from malicious programs
  - Allow multiple OS to coexist on a single host
    - Dedicated and customized images
  - ➢ There are other advantages, that will be relevant in next lesson
- Cons
  - Performance degradation: CPU (negligible) + I/O
  - Size of the virtual machine images

Contents

- Computer architecture
- Operating system functionality
- Access to the OS
- Memory and address spaces
- Execution of user programs
- Processes
- **Mass storage and file systems**

Mass storage

- Secondary storage is needed due to some limitations of main memory storage
  a) Too small to accommodate all data and programs
  b) Data is volatile
- Disk drives are used typically to store data in a permanent way
  – e.g. magnetic disks, solid-state disks
- OS is responsible for storage allocation, free-space management, disk scheduling, and disk access

Mass storage management

- Disk drives are addressed as large one-dimensional arrays of logical blocks
  - This facilitates the independence of the device
  - OS maps logical blocks into disk sectors
- OS accesses the device through the I/O bus using specific hardware operations (in/out)
  - Blocks are the smallest unit of transfer
- File systems provide convenient access to the disk by allowing easy data management
  - They define a logical storage unit: **file**

File system implementation

- File system implementation needs the OS to:

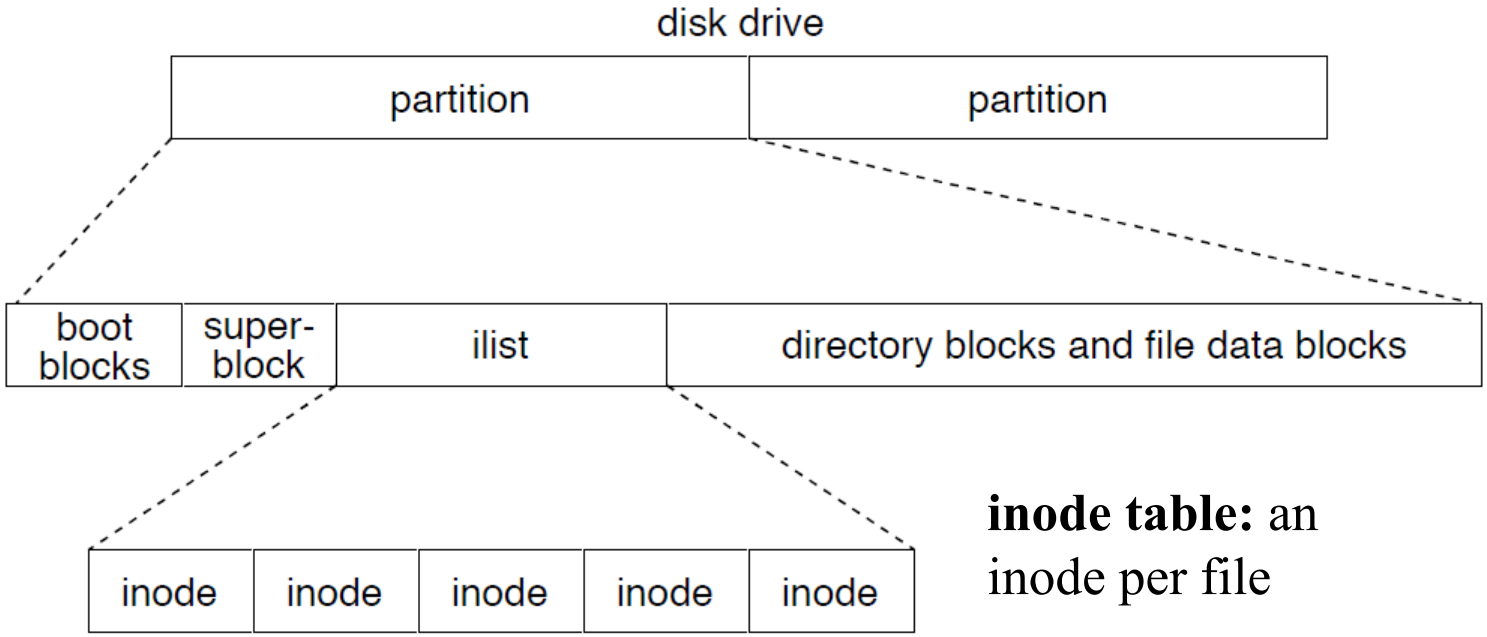
1. Define how file system looks to the user
   - Define a file and its attributes
   - Define the operations allowed on a file
   - Define the namespace: the organization of file names
   - Define the directory structure for organizing files

➢ This is independent on how & where files are stored on the disk

File system implementation

2. Define on-disk data structures to map the logical file system onto the physical disk

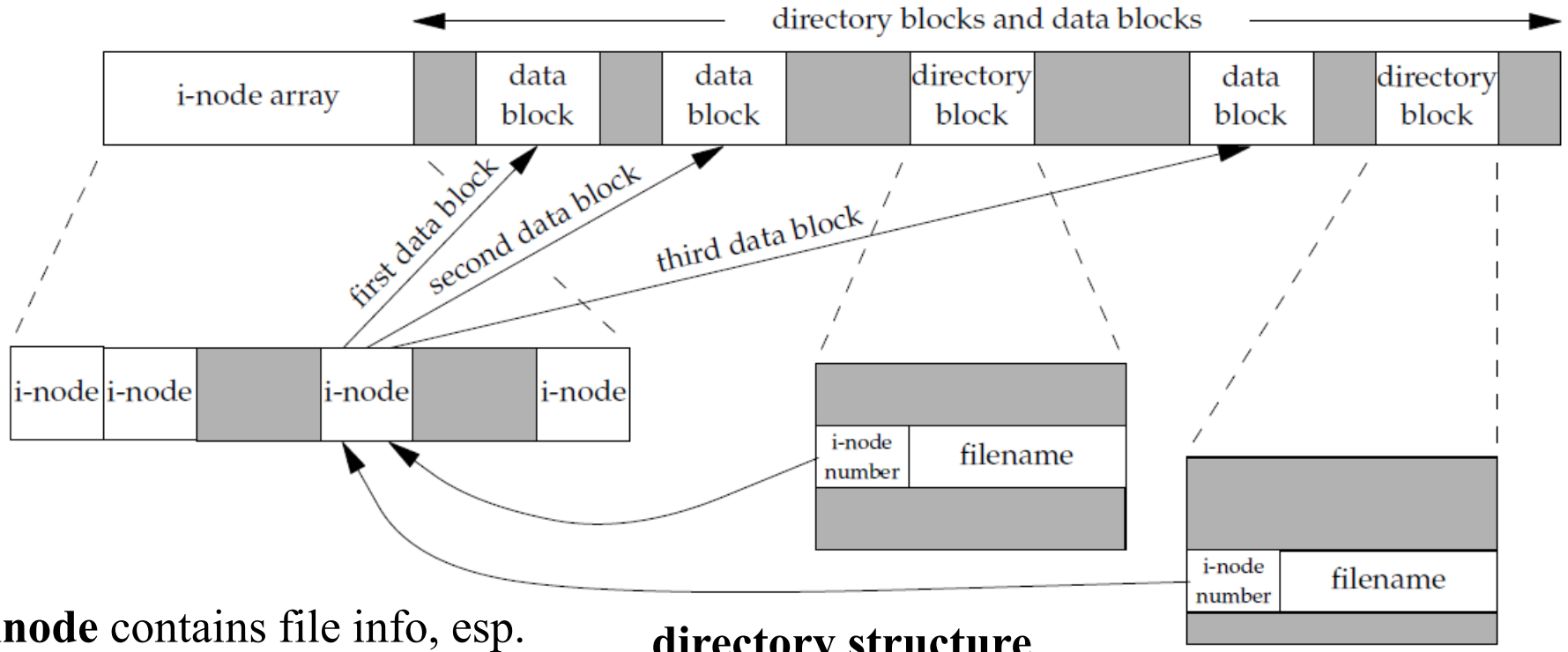
superblock: number (and size) of blocks and knowledge about free blocks/inodes, pointer to the root



boot block: code to boot an OS from that partition

inode table: an inode per file

File system implementation



inode contains file info, esp. which blocks the file uses

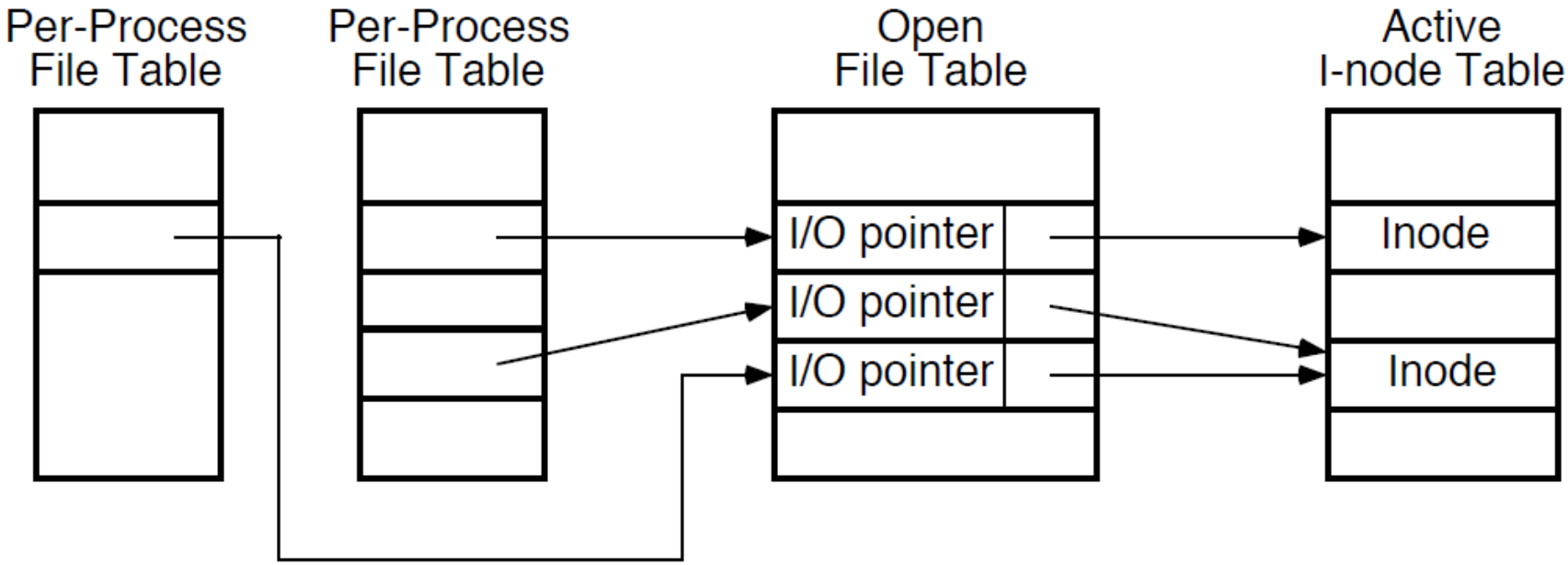
- Contiguous
- Linked list (FAT)
- Indexed (inodes)

directory structure including file names and associated inode numbers

Courtesy of [Stev13]

File system implementation

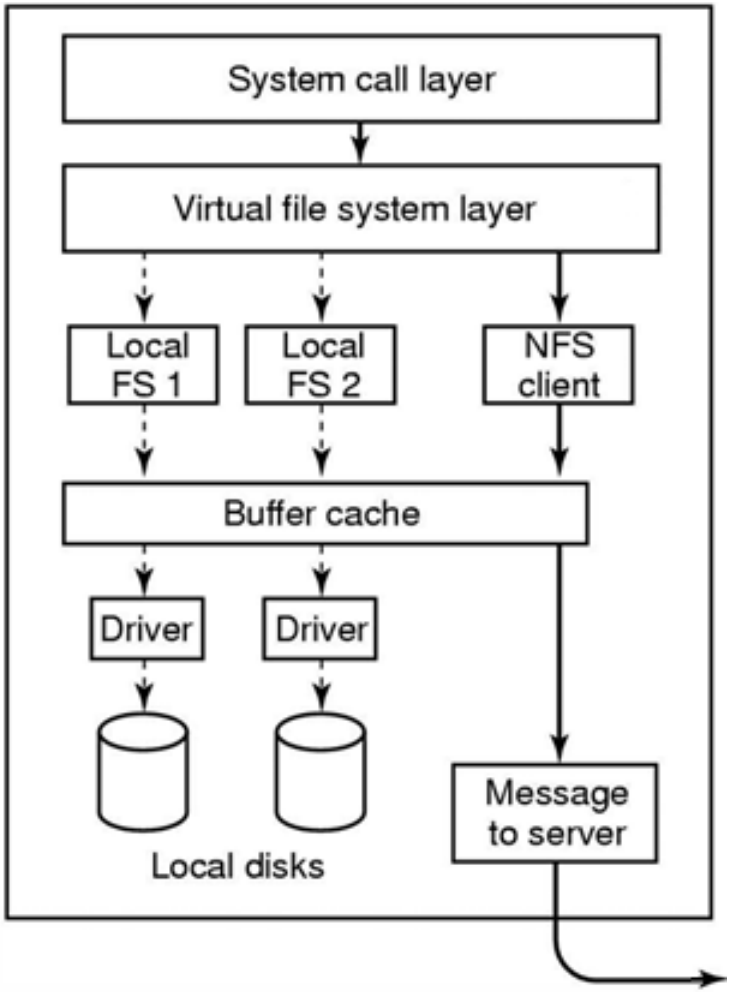
3. Define in-memory data structures for file management and performance improvement via caching



Courtesy of [Stev13]

File system implementation

4. Virtual File System (VFS) provides uniform and transparent access to local and remote files by separating file-system-generic operations from their implementation



Courtesy of [TaBo23]

File management

- OS is responsible for the following activities in connection with file management:
  - Creating and deleting files
  - Creating and deleting directories to organize files
  - Supporting primitives for manipulating files and directories
  - Mapping files onto mass storage
  - Backing up files on stable (nonvolatile) storage media

References

[SiGG18] Abraham Silberschatz, Peter Baer Galvin, and Greg Gagne, Operating System Concepts (10th ed.), Wiley Publishing, 2018

[Duar08] Gustavo Duarte, What Your Computer Does While You Wait, 2008, http://duartes.org/gustavo/blog/post/what-your-computer-does-while-you-wait/

[Stev13] W. Richard Stevens, Advanced Programming in the UNIX Environment 3rd ed., Addison Wesley, 2013

[TaBo23] A.S. Tanenbaum and H. Bos. 2023. Modern Operating Systems, 5th ed. Prentice Hall Press, Upper Saddle River, NJ, USA